

The Azums Product and Implementation Handbook

Version covered: Azums 1.0.0

Release baseline: annotated `v1.0.0` tag

Handbook date: 2026-08-16

This handbook explains what Azums is, why it exists, how its execution layer works, which features it provides, what its stable guarantees mean, what was tested for the 1.0 release, and where a new developer should begin when learning or integrating the project.

The most important source of truth is [Execution Semantics](#). When this handbook and a casual example appear to disagree, the semantic contract wins.

1. Executive Summary

Azums is a Rust-native asynchronous execution layer. It combines:

- a background job queue
- a Tokio worker runtime
- delayed and recurring scheduling
- leases, heartbeats, and crash recovery
- retries, failure classification, and a dead-letter queue
- idempotent enqueue
- same-database transactional enqueue on SQLite and PostgreSQL
- durable event streams with consumer-group offsets and replay
- runtime backend capability discovery
- job explanations, structured observations, and queue metrics
- adapters for Memory, SQLite, PostgreSQL, and Redis
- integrations for Axum, Actix Web, Poem, and Rocket

Azums is a library and execution runtime, not a hosted cloud service. An application embeds the client, registers handlers, and chooses the storage environment that provides the required durability and coordination.

The product is designed around one rule:

Every important behavior is classified as Guaranteed, Backend-dependent, or Unspecified.

Azums does not claim that every backend is operationally identical. It gives applications one job API while exposing transaction scope, durability, ordering, retention, notifications, backpressure, and distributed-worker support through `BackendCapabilities`.

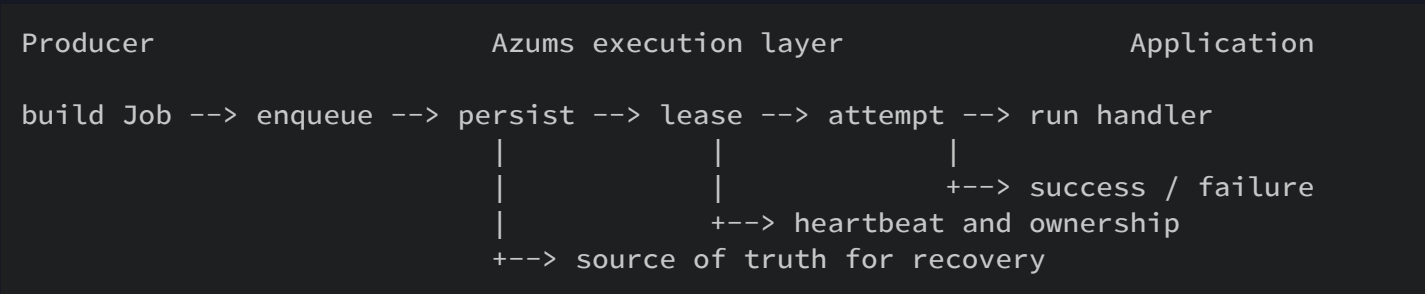
2. The Problem It Solves

A Rust application often needs to do work after the operation that requested it has finished:

- send an email after account creation
- process payments or webhooks
- generate reports and exports
- resize or transcode uploaded media
- call unreliable third-party APIs with retries
- index data after a database mutation
- execute long AI model or tool workflows
- process embedded-device telemetry
- publish domain events and rebuild projections

Executing this work inside a web request makes latency and availability depend on every downstream service. `tokio::spawn` moves the work out of the request, but an in-process future is lost when the process stops. A separate message broker can add durability, but it introduces transaction, recovery, retry, idempotency, deployment, and observability questions.

Azums turns the request into durable, inspectable work:

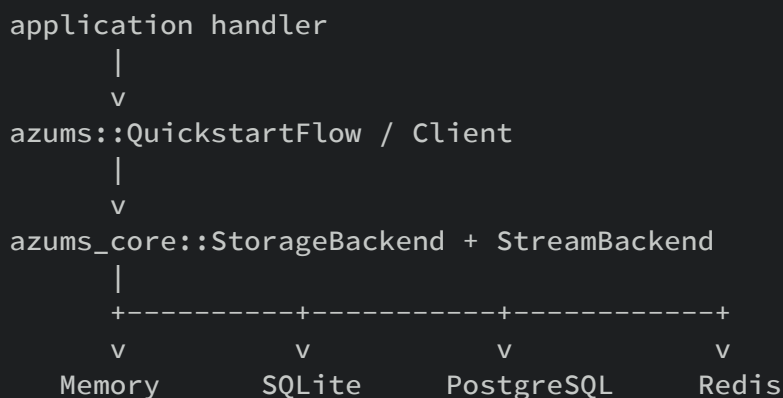


The execution layer owns delivery state. The application handler owns business effects. That boundary is essential: Azums can recover and redeliver a job, but it cannot make an arbitrary payment provider, email API, or filesystem mutation exactly once.

3. Workspace and Crate Map

Package	Responsibility
azums-core	Stable models, lifecycle semantics, backend traits, in-memory backend, stream and observability contracts.
azums	Main facade, quickstart client, Tokio worker runtime, SQLite/PostgreSQL adapters, repositories, migrations, CLI, tests, and benchmarks.
azums-postgres	PostgreSQL-oriented package and exports.
azums-redis	Redis job, lease, notification, stream, and offset implementation.
azums-axum	Axum state, extractor, and service integration.
azums-actix	Actix Web application-data and extractor integration.
azums-poem	Poem integration.
azums-rocket	Rocket managed-state/request-guard integration.
azums-dashboard	Optional and currently unstable dashboard/admin package.
worker	Example or standalone worker binary wiring handlers into the runtime.

The architectural dependency direction is:



Start with the facade in [quickstart.rs](#). Drop down to `StorageBackend` only when implementing a backend, using a backend-specific transaction, or building custom infrastructure.

4. Core Domain Model

Job

`Job` is the durable unit of requested work. It contains:

- `id` : generated UUID
- `dataset_id` : storage-routing identity
- `replay_of_job_id` : lineage when replay creates new work
- `idempotency_key` : producer-side duplicate protection
- `queue` : isolation and worker-selection boundary
- `job_type` : handler dispatch key
- `payload` : JSON application data
- `run_at` : first eligible execution time
- `deadline_at` : latest permitted start time
- `timeout_seconds` : maximum handler duration per attempt
- `recurring_interval_seconds` : fixed recurrence interval
- `status` : compact persisted status
- `priority` : higher values lease first where supported
- `max_attempts` : retry budget
- lease owner and timestamps
- DLQ reason and timestamp
- creation and update timestamps

JobAttempt

`JobAttempt` is the durable audit record of one handler invocation. It stores attempt ID and number, job ID, worker ID, start and finish timestamps, status, error code, error message, and latency. Failure belongs to an attempt; a retryable job itself returns to a future queued state.

JobExecution

`JobExecution` is the in-flight claim connecting a job, attempt, worker, lease expiration, and start time. It answers, "Who currently has the right to mutate this running job?"

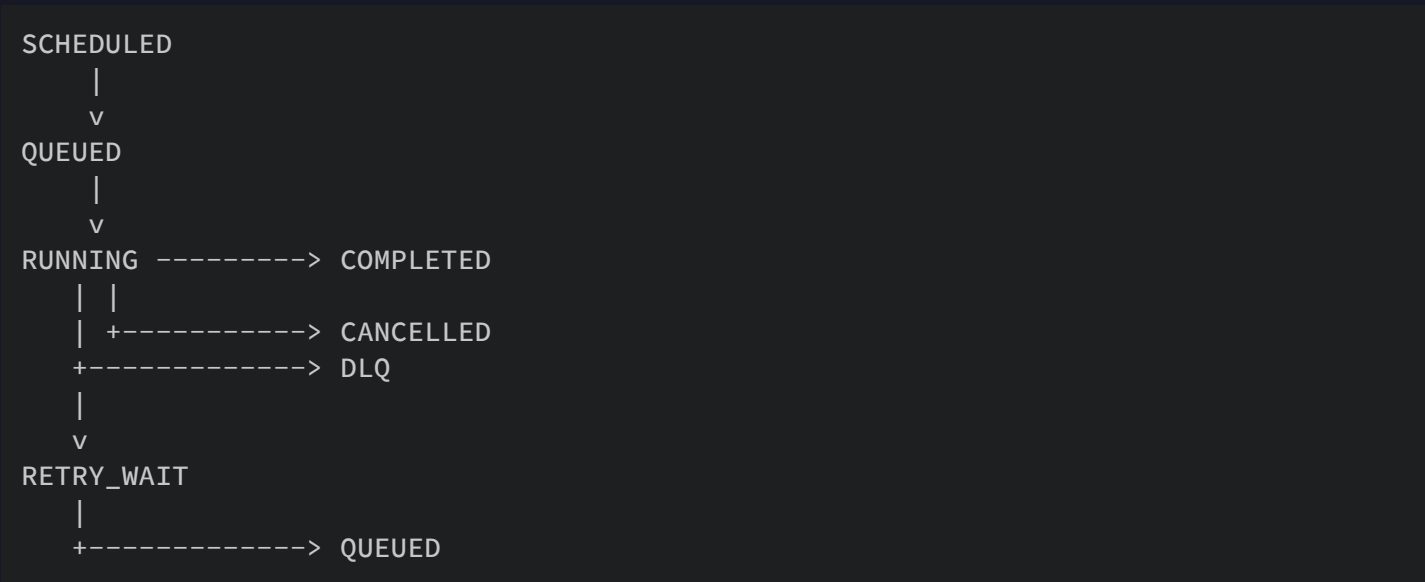
Worker and Queue

A worker identity owns leases and attempts. A queue selects which jobs a worker polls and provides an isolation boundary for ordering, policies, depth metrics, and scaling.

Event and Consumer Group

An `Event` is an immutable stream record with a stream-local monotonic sequence, event type, JSON payload, and timestamp. A consumer group stores its highest acknowledged sequence. Groups share the log but advance independently.

5. Canonical Job State Machine



Legal transitions are exactly:

Current state	Legal next state
SCHEDULED	QUEUED
QUEUED	RUNNING
RUNNING	COMPLETED , RETRY_WAIT , CANCELLED , or DLQ
RETRY_WAIT	QUEUED
COMPLETED	none
CANCELLED	none
DLQ	none; replay creates a new job instead

Every unlisted transition is illegal. `COMPLETED`, `CANCELLED`, and `DLQ` are terminal.

Storage keeps `SCHEDULED` and `RETRY_WAIT` compact: both use a queued job with future `run_at`. Lifecycle reconstruction distinguishes them using the future timestamp and whether prior failed attempts exist. The canonical rules are implemented by `JobLifecycleState` in [model.rs](#).

6. One Job's End-to-End Journey

This is the best path through the code for understanding how the execution layer works.

Step 1: The producer constructs work

The application creates a `Job`, usually through builder methods:

```
let job = azums::Job::new(
    "send_welcome_email",
    serde_json::json!({ "user_id": "user-123" }),
)
.queue("emails")
.priority(10)
.max_attempts(5)
.idempotency_key("welcome:user-123");
```

`Job::new` provides defaults; converting it to `NewJob` removes execution-owned fields before enqueue.

Step 2: The client selects a backend

`quickstart(url)` maps the URL to an adapter, runs storage setup or migrations, and returns a `QuickstartFlow`:

<code>memory</code>	<code>-> MemoryBackend</code>
<code>sqlite://jobs.db?mode=rwc</code>	<code>-> SqliteBackend</code>
<code>postgres://...</code>	<code>-> PostgresBackend</code>
<code>redis://... or rediss://...</code>	<code>-> RedisBackend</code>

Step 3: Enqueue becomes durable backend state

`QuickstartFlow::enqueue` calls `StorageBackend::enqueue`. The backend validates and persists the job, applies idempotency rules, and returns the logical job ID. Notifications are only wake-up hints; the stored job is the source of truth.

Step 4: A worker leases eligible jobs

`run` or `run_with_shutdown` repeatedly asks the backend for runnable work. Eligibility considers queue, status, `run_at`, deadline, ordering, priority, and backend policy. The lease records `locked_by` and `lock_expires_at`.

PostgreSQL coordinates leases using SQL row locking such as `FOR UPDATE SKIP LOCKED`. SQLite uses embedded SQL transactions, Redis uses atomic Redis operations, and Memory uses an in-process lock.

Step 5: The runtime starts attempts

Before calling handlers, `start_attempts_batch` creates durable attempt records. Starting an attempt requires the job to be running under the requesting worker's lease. This prevents a worker from manufacturing an attempt for work it does not own.

Step 6: Dispatch and heartbeat

The runtime finds the registered handler by `job_type`. While the handler runs, a heartbeat task tries to extend the lease every half-lease interval. Missing handlers become classified failures. Handlers run in Tokio tasks so panics can be isolated and recorded rather than terminating the entire worker loop.

Step 7: Success, failure, or timeout

- Success closes the attempt and ACKs the job as completed.
- A timeout aborts the handler task and enters normal retry/DLQ classification as `TIMEOUT`.
- A panic is captured with available panic information and routed to DLQ.
- Retryable and system failures are rescheduled with backoff and jitter while attempts remain.
- Permanent failures and exhausted retries enter DLQ.

Every mutation of running work checks worker ownership. Completion cannot be applied by a worker that no longer owns the lease.

Step 8: Recovery after disappearance

Workers periodically reap expired leases. Recovery records the abandoned attempt as `LEASE_EXPIRED`, clears lease ownership, and makes the job eligible again according to retry rules. The committed job does not silently vanish.

Because a handler may have completed its external effect before its process died, recovery can cause duplicate execution. This is expected at-least-once behavior.

Step 9: Explanation and operations

`explain_job(job_id)` returns the current status, retries, last worker, error, trace ID, event timeline, and a human-readable summary where the backend has native observability. `queue_metrics` and `metrics_snapshot` expose queue-level counters and latency summaries.

7. Feature Catalog

Job production

- individual and batch enqueue
- named job types and queues
- arbitrary JSON payloads and typed deserialization
- priorities and queue ordering configuration
- idempotency keys
- payload and producer-rate guards
- replay with lineage

Scheduling

- immediate execution
- absolute `run_at`
- relative delay
- execution deadline

- per-attempt timeout
- retry delay, exponential backoff, cap, and jitter
- fixed-interval recurring execution
- deterministic recovery of scheduled jobs after downtime

Worker execution

- closure handlers and trait-based `JobProcessor` handlers
- batch leasing
- worker identities
- exclusive leases
- background heartbeat extension
- graceful shutdown with `CancellationToken`
- periodic expired-lease recovery
- notification-driven wake-up with polling fallback
- periodic backend maintenance

Failure handling

- retryable error
- permanent error
- timeout
- panic isolation
- cancellation
- system failure
- configurable maximum attempts
- dead-letter queue inspection
- durable errors, reasons, workers, timings, and attempts where retained
- DLQ replay

Coordination and overload

- concurrent workers
- queue isolation
- priority and FIFO leasing where declared
- PostgreSQL execution-rate queue policies
- backlog-only behavior for Memory, SQLite, and Redis
- queue policy and ingest decision records
- producer storm controls

Streams

- append-only events
- monotonic per-stream sequence numbers
- offset reads
- independent consumer-group offsets
- monotonic ACK
- replay without implicit ACK
- notification subscription
- retention-aware pruning that respects the slowest known group offset

Observability and operations

- job lookup and lifecycle explanation
- attempt and worker history
- structured job log events
- queue metrics
- retry, DLQ, execution, and claim information
- maintenance and archive support
- CLI tools including `azumsctl`, `azums-perf`, and `azums-perf-guard`
- optional admin/dashboard package
- deployment guide, recovery runbook, benchmark harness, and architecture book

8. Backend Capability Matrix

Capability	Memory	SQLite	PostgreSQL	Redis
Portable job API	Yes	Yes	Yes	Yes
Idempotent enqueue	Process-local	Unique index	Unique index	Redis hash
Same-app-data transaction	No	Same SQLite DB	Same PostgreSQL DB	No
Job durability	No	File-backed DB	Persistent DB	Depends on Redis configuration
Notifications	In-process	In-process plus polling	LISTEN/NOTIFY hint	Pub/Sub plus polling

Capability	Memory	SQLite	PostgreSQL	Redis
Streams	Yes	Yes	Yes	Yes
Consumer-group offsets	Process-local	SQL	SQL	Redis hash
Distributed workers	No	No	Yes	Yes
Ordering	FIFO and fastest modes	FIFO and fastest modes	FIFO and fastest modes	FIFO leasing
Backpressure	Backlog	Backlog	Execution-rate policies	Backlog
Retention	Process lifetime	Explicit maintenance	Explicit maintenance	Backend configured

Selection guidance:

- Choose **Memory** for tests and short-lived local work.
- Choose **SQLite** for durable embedded, desktop, edge, CLI, or single-binary applications.
- Choose **PostgreSQL** when application mutations and jobs must share a transaction or workers run across hosts.
- Choose **Redis** for Redis-native distributed execution when Redis persistence and eviction are configured appropriately and SQL transaction coupling is not required.

Production code should inspect capabilities instead of assuming them:

```
let client = azums::quickstart(std::env::var("DATABASE_URL"?).await?;
let caps = client.capabilities();

anyhow::ensure!(caps.durable_jobs, "production requires durable jobs");
anyhow::ensure!(
    caps.distributed_workers,
    "this deployment runs workers on multiple hosts"
);
```

9. What Does Azums Promise?

Guaranteed across the portable contract

- Successfully enqueued, runnable, non-cancelled work is delivered at least once while the chosen backend retains it and workers are available.

- Illegal lifecycle transitions are rejected.
- Terminal states remain terminal.
- A job has at most one valid active lease.
- Expired leases are recoverable.
- Attempt numbers and consumer offsets do not move backward.
- Failure classification determines retry or DLQ behavior consistently.
- A non-null idempotency key identifies one logical job.
- Jobs are not intentionally leased before scheduling eligibility according to the backend clock.
- Stream sequence numbers increase monotonically within a stream.
- Replay creates new work with lineage and preserves original history.
- Cancellation follows lease ownership and terminal-state rules.

Backend-dependent

- durability through process, machine, or storage failure
- job, attempt, and stream retention
- transaction scope
- notification behavior and wake-up latency
- distributed worker coordination
- ordering strength
- backpressure enforcement
- Redis persistence and eviction behavior
- backend clock authority and scheduling precision

Explicitly not guaranteed

- exactly-once job execution
- exactly-once external side effects
- execution at the exact `run_at` instant
- global ordering or completion ordering
- worker fairness
- automatic consumer-group member assignment or balancing
- transactions across arbitrary external services
- permanent retention
- automatic scaling, producer blocking, or load shedding
- calendar or daylight-saving-aware recurrence
- generation of every missed recurring occurrence after downtime
- cancellation undoing an external effect that already occurred

10. Questions Azums Can Answer

Question	Precise answer
Can a committed job silently disappear?	A retained job on a correctly configured durable backend is not intentionally discarded. A crashed worker's lease expires and the job becomes recoverable. Memory and misconfigured Redis do not provide the same durability.
Is execution exactly once?	No. Delivery is at least once. A crash after side effect but before ACK can cause another execution.
How do I make side effects safe?	Use application-level idempotency keyed by job ID, idempotency key, or stream sequence, or use a same-database transaction/outbox boundary.
Will a scheduled job run exactly on time?	It will not intentionally lease before eligibility. Exact start time is not guaranteed.
What happens when a handler fails?	The error is classified. Retryable failures wait according to policy; permanent failures, panics, and exhausted attempts go to DLQ.
What happens when a worker dies?	Heartbeats stop, the lease expires, recovery records abandonment, and the job can become executable again.
Can two workers hold the same lease?	The contract permits at most one valid active lease. Backend-specific atomic claim mechanisms enforce it.
Does cancellation stop a running external API call?	Cancellation changes Azums state according to ownership rules; it cannot undo an already completed external effect.
Does replay retry the same historical object?	Job replay creates a new job with lineage. Stream replay reads history. Neither erases original history.
Can Redis enqueue atomically with PostgreSQL app data?	No. Redis is atomic inside Redis, not inside an unrelated SQL transaction.
Which stream event does a group receive next?	The first retained event with <code>sequence_no > last_acked_seq</code> for that group.
Do consumer groups divide work automatically?	No. Azums 1.0 provides durable offsets, not automatic membership or balancing.
What happens under overload?	Memory, SQLite, and Redis accept backlog according to capacity. PostgreSQL can throttle execution leases. Azums does not automatically scale or silently invent an admission policy.

Question	Precise answer
Is completion order FIFO?	No. Ordering affects lease selection. Different handler durations and retries can change completion order.
How long is history retained?	Backend-dependent. Inspect retention capabilities and configure maintenance explicitly.

11. Transactional Enqueue

For SQLite and PostgreSQL, application data and job data can use one transaction:

```
use azums::{Job, PostgresBackend};
use serde_json::json;

async fn create_user(
    pool: &sqlx::PgPool,
    backend: &PostgresBackend,
) -> anyhow::Result<()> {
    let mut tx = pool.begin().await?;

    sqlx::query("INSERT INTO users (id) VALUES ($1)")
        .bind("user-123")
        .execute(&mut *tx)
        .await?;

    backend
        .enqueue_in_tx(
            &mut tx,
            Job::new(
                "send_welcome_email",
                json!({ "user_id": "user-123" }),
            )
            .into(),
        )
        .await?;

    tx.commit().await?;
    Ok(())
}
```

A successful commit preserves both records. Rollback, transaction drop before commit, deferred commit failure, connection loss before commit, and process termination before commit preserve neither. PostgreSQL notifications issued in the transaction are visible only after commit. SQLite workers discover committed jobs through their polling fallback.

12. Idempotency and Duplicate Delivery

Idempotent enqueue and idempotent execution solve different problems:

```
100 enqueue calls + same idempotency key
    -> one logical job

one logical job + crash before ACK
    -> handler may run again
```

For an external API, pass a stable key derived from the logical operation:

```
let external_key = format!("azums-job:{}", job.id);
payment_api.charge_with_idempotency_key(external_key, amount).await?;
```

For a database effect, record processed job IDs or stream sequences under a unique constraint in the same transaction as the effect. A duplicate delivery then becomes a no-op.

13. Durable Event Streaming

```
use azums::{quickstart, NewEvent};
use serde_json::json;

async fn stream_example() -> anyhow::Result<()> {
    let client = quickstart("memory").await?;
    let orders = client.stream("orders");

    let seq = orders
        .publish("order_created", json!({ "order_id": "ord-1001" }))
        .await?;

    let events = orders.read_next("billing", 100).await?;
    for event in events {
        // Make the external effect idempotent by sequence number.
        orders.ack("billing", event.sequence_no).await?;
    }

    println!("published sequence {seq}");
    Ok(())
}
```

`read_next` begins after the group's durable ACK. Reading does not advance the offset; successful processing must ACK. If the consumer crashes before ACK, the event is delivered again. Retention pruning cannot delete beyond the lowest known group offset.

14. Observability

The application can ask Azums to explain a job instead of reading internal tables:

```
if let Some(explanation) = client.explain_job(job_id).await? {
    println!("{}", explanation.summary);
    println!("status: {}", explanation.status);
    println!("retries: {}", explanation.retry_count);
    println!("last worker: {:?}", explanation.last_worker_id);
    println!("last error: {:?}", explanation.last_error);
}

for metrics in client.metrics_snapshot().await? {
    println!("{}", depth={}, metrics.queue, metrics.queue_depth);
}
```

Stable observation fields include job ID, attempt, worker ID, queue, duration, status, retry count, error, and trace ID. Metrics cover totals, completions, failures, retries, DLQ count, queue depth, latencies, and worker count where the backend can measure them. Backends without native attempt observability return a reduced explanation from the current job state rather than fabricating data.

15. Test and Release Evidence

The release-candidate record for the exact 1.0.0 tree states that no known documented guarantee was violated. It is evidence from a specific tree and environment, not a mathematical proof that all future programs or infrastructure failures are harmless.

Release gates run

Area	Evidence recorded for 1.0.0
Full workspace	<code>cargo test --workspace</code> passed.
Integration	Included in workspace tests; PostgreSQL and Redis profiles ran when services were reachable.
State invariants	Core unit tests and generated lifecycle transition properties passed.
Lease recovery	Lease expiry, worker crash, heartbeat, phantom recovery, and wrong-owner paths passed.
Transactions	SQLite/PostgreSQL commit and rollback boundaries, commit failure, connection loss, and process termination passed.

Area	Evidence recorded for 1.0.0
Failure handling	Retry, classification, timeout, panic, cancellation, DLQ, and replay suites passed.
Idempotency	Duplicate enqueue and crash-after-side-effect patterns passed.
Scheduling	Eligibility, downtime, skew-sensitive boundaries, deadlines, timeouts, and recurrence suites passed.
Streams	Append, independent offsets, restart, replay, duplicates, ordering, concurrency, and retention passed.
Property tests	M12 generated job sequences, transitions, retries, schedules, duplicates, workers, and rollback combinations passed.
Fuzz hardening	M13 malformed payload, metadata, type, queue, serialization, event, and API-boundary tests passed.
Chaos	Standard chaos suite plus 10,000 randomized memory scenarios passed.
Large concurrency	24 combinations of 1, 2, 5, 10, 50, and 100 workers with 10k, 50k, 100k, and 1m jobs passed.
Scale total	The final matrix executed 6.96 million jobs in 219.27 seconds on the recorded release machine.
Benchmarks	Criterion suites and the reproducible M14 harness passed; Redis Criterion remained opt-in.
Regression guard	M15 throughput, latency, and measured resource regression logic passed.
Documentation	<code>mdbook build docs</code> passed with previously documented benchmark HTML warnings.
Security audit	<code>cargo audit</code> reported zero active shipped-path vulnerabilities against 1,216 advisories on 2026-08-15; an inactive optional MySQL RSA lockfile dependency was scoped and documented.
API compatibility	API audit, matrix guard, doctests, and the declared <code>v0.2.0</code> to <code>1.0.0</code> semver-major transition check passed.

What was not turned into a universal claim

- Redis-specific Criterion throughput is opt-in and requires controlled Redis infrastructure.
- Live PostgreSQL/Redis database restarts, network partitions, and process-death profiles depend on infrastructure capable of injecting those failures.
- Benchmark throughput is workload, hardware, backend, configuration, and version specific.

- Static and generated tests reduce risk; they do not prove that arbitrary external side effects are exactly once.

Test sources are under [crates/azums/tests](#), with core properties under [crates/azums-core/tests](#). The summarized proof is in [M20 Release Candidate Evidence](#).

16. Implementing Azums in a New Application

Step 1: Add dependencies

```
cargo add azums@1.0.0 anyhow serde serde_json  
cargo add tokio --features macros,rt-multi-thread
```

Enable or select the storage features required by your application according to the published crate's feature list.

Step 2: Begin in memory

```
use azums::{quickstart, Job};
use serde::Deserialize;
use serde_json::json;

#[derive(Deserialize)]
struct EmailPayload {
    to: String,
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let client = quickstart("memory").await?.with_queue("emails");

    client
        .register_handler("send_email", |job| async move {
            let payload: EmailPayload = job.payload_typed()?;
            println!("send to {}", payload.to);
            Ok(())
        })
        .await;

    let id = client
        .enqueue(
            Job::new("send_email", json!({ "to": "user@example.com" })))
            .queue("emails")
            .max_attempts(5)
            .idempotency_key("welcome:user@example.com"),
        )
        .await?;

    client.run_until_empty().await?;
    println!("{:?}", client.explain_job(id).await?);
    Ok(())
}
```

Run the tested repository example with:

```
cargo run -p azums --example install_enqueue_process_retry_inspect
```

Step 3: Move to SQLite for local durability

Change only the URL:

```
let client = quickstart("sqlite://jobs.db?mode=rwc").await?;
```

Use this for embedded applications, developer machines, CLIs, and single-process services. Verify that the database file is on durable storage and include it in backup and disk-capacity planning.

Step 4: Move to PostgreSQL or Redis for distributed workers

PostgreSQL is the default recommendation when application rows and jobs must commit together. Redis is suitable when the deployment is Redis-native and its persistence/eviction configuration matches the application's durability requirements.

Give every process a unique worker ID and select its queue:

```
AZUMS_WORKER_ID=worker-email-01
AZUMS_QUEUE=emails
AZUMS_LEASE_SECONDS=30
```

Step 5: Run continuously with graceful shutdown

```
let shutdown = tokio_util::sync::CancellationToken::new();
let worker = client.clone();
let worker_shutdown = shutdown.clone();

let task = tokio::spawn(async move {
    worker.run_with_shutdown(worker_shutdown).await
});

// On SIGTERM or application shutdown:
shutdown.cancel();
task.await??;
```

Set lease duration longer than normal heartbeat and handler scheduling delays but shorter than the maximum acceptable crash-recovery time. Handler timeouts and lease duration solve different problems: timeout bounds one attempt; the lease decides when another worker may recover abandoned work.

Step 6: Design side-effect idempotency

Before production, answer:

1. What happens if this handler runs twice?
2. Can the destination accept an idempotency key?
3. Can a processed-job record and database mutation share a transaction?

4. Is replay allowed to repeat the effect?

Do not deploy a payment, email, webhook, or provisioning handler until duplicate behavior is deliberate.

Step 7: Add operations

- alert on queue depth, retries, DLQ, execution latency, and lease expiry
- set payload and enqueue-rate limits
- protect the admin API with authentication, TLS, and network restrictions
- run migrations as a controlled deployment step
- canary new worker versions
- back up durable storage
- document replay authorization
- rehearse database and worker failure recovery

17. How to Learn the Repository

Beginner path: use the product first

1. [Zero-Config Quickstart](#)
2. [Developer Experience](#)
3. [install_enqueue_process_retry_inspect.rs](#)
4. [embedded_sqlite.rs](#)
5. [Execution Semantics](#)

At this stage, be able to enqueue, register a handler, process, retry, inspect, and explain why delivery is at least once.

Application engineer path: integrate safely

1. [Storage Backend Equivalence](#)
2. [Transactional Integrity](#)
3. [Idempotency](#)
4. [Lease Recovery](#)
5. [Failure Handling](#)
6. [Scheduling and Time](#)
7. [Observability](#)

8. [Production Deployment](#)
9. [Failure and Recovery Runbook](#)

At this stage, be able to choose a backend, define transaction and idempotency boundaries, tune a lease, handle shutdown, and operate DLQ/replay safely.

Maintainer path: follow one job through the code

1. Models and state rules: [azums-core/src/model.rs](#)
2. Machine-readable contract: [azums-core/src/semantics.rs](#)
3. Backend interface: [azums-core/src/backend/mod.rs](#)
4. Beginner facade and worker loop: [azums/src/quickstart.rs](#)
5. PostgreSQL job mutations: [azums/src/jobs/repo.rs](#)
6. Attempt records: [azums/src/jobs/attempts.rs](#)
7. Retry decisions: [azums/src/jobs/retry.rs](#)
8. SQLite adapter: [azums/src/backend/sqlite.rs](#)
9. PostgreSQL adapter: [azums/src/backend/postgres.rs](#)
10. Redis adapter: [azums-redis/src/backend.rs](#)
11. Stream contract and facade: [stream.rs](#) and [stream_handle.rs](#)
12. Invariant tests: [core_unit.rs](#), [lease_recovery.rs](#), and [m12_property_based.rs](#)

Read one backend mutation and its corresponding tests together. For example, read lease claim, attempt creation, ACK, retry, and expired-lease reaping as one lifecycle rather than isolated SQL methods.

Backend implementer path

A custom backend implements `StorageBackend` and optionally `StreamBackend` and `ObservabilityBackend`. It must:

- declare truthful capability flags and detailed semantics
- make claims atomic
- enforce lease ownership on running mutations
- reject terminal-state mutation
- preserve monotonic attempts and offsets
- make expired work recoverable
- ensure notifications remain hints rather than the source of truth
- implement idempotency in the documented scope
- add unit, integration, concurrency, failure, and backend matrix tests

Do not copy a capability profile unless the backend genuinely provides every declared behavior.

18. Documentation Map

Need	Start here
What is Azums?	This handbook and Architecture Overview
What is guaranteed?	Execution Semantics
Which backend should I use?	Backend Equivalence
How do jobs transition?	Job Lifecycle
How do retries and DLQ work?	Failure Handling
How do leases recover crashes?	Lease Recovery
How do SQL transactions work?	Transactional Integrity
How do duplicates work?	Idempotency
How does scheduling behave?	Time Semantics
How do streams and offsets work?	Durable Event Streaming
How does overload behave?	Concurrency and Backpressure
How do I observe failures?	Observability
How was it tested?	Primitive Correctness , Chaos , Property Testing , Fuzzing , and Release Evidence
How do I deploy it?	Production Deployment
What do I do during an incident?	Failure and Recovery Runbook
Which APIs are stable?	API Stability Policy and Stable Release Gate

19. Production Checklist

Before adopting Azums in production, verify:

- the backend capability profile satisfies your durability and coordination needs
- every worker has a unique ID
- queue names and queue ownership are documented

- lease duration, handler timeout, retry budget, and deadline are independently configured
- external side effects are idempotent
- SQL transactional enqueue is used where app/job divergence would be harmful
- Redis persistence and eviction are intentional, if Redis is selected
- SQLite is not being used as an undocumented multi-host coordinator
- payload and enqueue-rate limits are configured
- admin access is authenticated and network restricted
- migrations and rollback procedures are reviewed
- queue depth, retry, DLQ, latency, worker, and lease-expiry alerts exist
- operators know when replay is safe
- retention, pruning, archive, and backups are configured
- graceful shutdown and forced-kill recovery have been rehearsed
- the application's own failure scenarios are tested in addition to Azums' suite

20. Final Mental Model

Think of Azums as three related layers:

1. Durable intent
Job, schedule, priority, idempotency, stream event
2. Controlled execution
claim, lease, heartbeat, attempt, handler, ACK, retry, DLQ
3. Explanation and recovery
persisted history, offsets, replay, metrics, lifecycle explanation

The durable record is the source of truth. Notifications make workers faster but do not define correctness. Leases make abandoned work recoverable but imply at-least-once delivery. Idempotency protects logical production and application-designed side effects. Backend capabilities tell the application which operational promises are actually available.

That is the purpose of the Azums execution layer: allow Rust applications to move work out of the immediate request path without losing a precise answer to what happened, what can happen next, and what the system promised.